

Relatório de Laboratório

Laboratório de Experimentação de Software

Curso	Engenharia de Software
Disciplina	Laboratório de Experimentação de Software
Turno / Período	Noite / 6º
Professor(a)	Danilo Maia
Laboratório	Lab01 — Características de Repositórios Populares + Setup do Kanban
Grupo (trio)	João Gabriel Santos Andrade Almeida · Lucas Ferreira Ribeiro · Vítor Veiga Silva
Link do repositório / GitHub Projects	https://github.com/JoaoAlmeida02/Laborat-rio-de-Experimenta-o-de-Software
Data de entrega	26/08/2026

1. Introdução

O ecossistema de software de código aberto (open source software, OSS) tornou-se uma parte fundamental da engenharia de software atual. De acordo com o GitHub Octoverse 2025 (GITHUB, 2025), a plataforma ultrapassou 180 milhões de desenvolvedores e 395 milhões de repositórios públicos. Além disso, o crescimento do uso de Inteligência Artificial e de ferramentas generativas tem acelerado ainda mais a criação e a colaboração em projetos de código aberto. Nesse contexto, entender as características dos repositórios mais populares e a forma como são mantidos é importante para analisar a evolução e a sustentabilidade desse ecossistema.

Neste laboratório, analisamos uma amostra formada pelos 1.000 repositórios com maior número de estrelas no GitHub. A partir dos dados do GitHub Octoverse 2025, foram definidas as seguintes Questões de Pesquisa (RQs) e hipóteses informais.

RQ01. Sistemas populares são maduros/antigos?

Hipótese informal (H1): Espera-se que a maioria dos repositórios populares tenha uma idade elevada, com mediana superior a 5 anos. A ideia é que projetos populares geralmente precisam de tempo para construir uma comunidade, ganhar visibilidade e formar uma base de usuários.

RQ02. Sistemas populares recebem muita contribuição externa?

Hipótese informal (H2): Considerando o crescimento de 20,4% no número de pull requests criadas e os mais de 518 milhões de PRs mesclados em 2025, espera-se que os repositórios populares apresentem uma quantidade elevada de pull requests aceitas. A grande visibilidade desses projetos pode atrair mais desenvolvedores externos.

RQ03. Sistemas populares lançam releases com frequência?

Hipótese informal (H3): Espera-se que projetos populares e consolidados tenham uma frequência significativa de releases. Isso pode indicar uma preocupação constante com a evolução e a entrega de novas versões do software.

RQ04. Sistemas populares são atualizados com frequência?

Hipótese informal (H4): Como o GitHub registrou quase 1 bilhão de commits em 2025, espera-se que os repositórios mais populares também apresentem atualizações frequentes. Assim, o tempo desde a última atualização, representado pelo atributo pushedAt, deve ser baixo, com uma mediana de poucos dias.

RQ05. Sistemas populares são escritos nas linguagens mais populares?

Hipótese informal (H5): Considerando o ranking do GitHub Octoverse 2025, no qual TypeScript aparece como a linguagem mais utilizada, seguida por Python, JavaScript e Java, espera-se que essas linguagens estejam entre as mais presentes nos 1.000 repositórios mais estrelados.

RQ06. Sistemas populares possuem um alto percentual de issues fechadas?

Hipótese informal (H6): Considerando o crescimento de 11,3% no fluxo de issues em 2025, espera-se que os repositórios populares apresentem uma alta proporção de issues fechadas. A hipótese é que a taxa de resolução, calculada pela relação entre issues fechadas e o total de issues, seja superior a 70%, indicando uma boa capacidade de manutenção e organização dos projetos.

RQ07. Sistemas escritos em linguagens mais populares recebem mais contribuição externa, lançam mais releases e são atualizados com mais frequência?

Hipótese informal (H7): Espera-se que repositórios escritos em linguagens como TypeScript, Python e JavaScript apresentem uma quantidade maior de pull requests aceitas e uma frequência maior de atualizações quando comparados a projetos desenvolvidos em linguagens menos populares. Isso pode estar relacionado à maior quantidade de desenvolvedores e à maior utilização dessas tecnologias.

Inovações Propostas pelo Grupo:

RQ08 (Bônus / Inovação): As estrelas estão relacionadas ao engajamento real dos projetos ou representam apenas popularidade?

Hipótese informal (H8): Espera-se que a relação entre o número de estrelas (*stargazers*) e as métricas de engajamento ativo, como o total de PRs aceitas e o número de issues movimentadas, seja fraca a moderada. Isso ocorre porque receber muitas estrelas nem sempre significa que o projeto possui uma comunidade ativa. Em muitos casos, as estrelas funcionam mais como uma forma de marcar um repositório ou demonstrar interesse nele. Dessa forma, espera-se que apenas uma parte dos projetos mais populares também apresente altos níveis de contribuição e manutenção ativa.

2. Contexto

Este trabalho corresponde ao Laboratório 01 da disciplina de Laboratório de Experimentação de Software, do 6º período, no turno da noite, sob orientação do Prof. Danilo Maia. O laboratório inicia a parte metodológica do semestre, unindo a coleta de dados por meio de um script próprio, que utiliza a API GraphQL do GitHub, com a organização das atividades no quadro Kanban do GitHub Projects v2.

O objeto de estudo é uma amostra dos 1.000 repositórios com maior número de estrelas (*stargazers*) no GitHub. Embora as estrelas sejam frequentemente utilizadas em estudos de Mineração de Repositórios de Software (MSR) como uma forma de medir a popularidade dos projetos, a inclusão da RQ08 busca analisar se esse indicador realmente está relacionado ao nível de atividade e engajamento dos repositórios.

Para manter um padrão na análise, o relatório GitHub Octoverse 2025, intitulado A new developer joins GitHub every second as AI leads TypeScript to #1, será utilizado como referência para definir as linguagens de programação mais populares mencionadas ao longo do trabalho. A base conceitual e formal utilizada para as medições segue o framework de medição de software apresentado por Zuse (2013).

3. Metodologia

3.1 Principais Desafios

Durante o planejamento e a execução do laboratório, o grupo encontrou alguns desafios relacionados tanto à coleta dos dados quanto à organização das atividades no projeto.

Um dos principais problemas foi o gerenciamento do limite de requisições da API GraphQL do GitHub. A API possui uma cota de 5.000 pontos por hora para requisições autenticadas. Como era necessário coletar informações de 1.000 repositórios, incluindo dados como pull requests mescladas, releases e issues, foi necessário otimizar a consulta GraphQL. Também foi implementado um tratamento de erros com espera automática, conhecido como backoff, para lidar com possíveis respostas de erro, como os códigos HTTP 403 e 502.

Outro desafio foi a paginação dos dados. A API GraphQL permite retornar no máximo 100 registros por requisição. Por isso, foi necessário utilizar a paginação por cursores, através do parâmetro `after: $cursor`, para percorrer todos os resultados até chegar aos 1.000 repositórios. Esse processo foi importante para evitar que registros fossem perdidos durante a coleta.

Também foi necessário lidar com a variedade de tipos de projetos presentes entre os repositórios mais populares do GitHub. A amostra possui projetos tradicionais de software, mas também inclui tutoriais, livros, listas de recursos e outros tipos de repositórios. Alguns deles não possuem uma linguagem primária definida (`primaryLanguage == null`) ou utilizam issues e releases de maneira diferente de projetos convencionais.

3.2 Tomadas de Decisão

Uma das decisões do grupo foi desenvolver o próprio script utilizado na coleta dos dados. Conforme solicitado no enunciado, não foram utilizadas bibliotecas ou SDKs de terceiros para abstrair o acesso à API do GitHub, como PyGithub ou Octokit. A comunicação com a API GraphQL e a autenticação por meio de Personal Access Token (PAT) foram implementadas diretamente no código desenvolvido pelo grupo.

Também foi definido um limite de WIP (Work in Progress) igual a 3 para a coluna Doing do quadro Kanban. Como o grupo é formado por João Gabriel, Lucas Ferreira e Vítor Veiga, cada integrante pode ficar responsável por uma tarefa em andamento por vez. Essa organização ajuda a evitar o excesso de tarefas simultâneas e facilita o acompanhamento do trabalho.

Os repositórios que não possuem uma linguagem primária definida foram mantidos no conjunto de dados para as análises gerais das RQs 01 a 04 e da RQ06. Porém, esses registros foram separados nas análises relacionadas às linguagens, principalmente nas RQs 05 e 07, para evitar que valores sem classificação interferissem nos resultados.

Outra decisão foi utilizar a mediana como principal medida de tendência central. Os dados dos repositórios apresentam diferenças muito grandes entre os projetos e possuem alguns valores extremos. Nesse cenário, a mediana representa melhor o comportamento geral da amostra do que a média.

3.3 Etapas

O desenvolvimento do laboratório foi organizado em três sprints, seguidas pela elaboração do relatório final. As atividades foram divididas em Issues no GitHub Projects e atribuídas aos integrantes do grupo.

Sprint / Etapa	Entregas realizadas	Responsável principal	Issues associadas
Lab01S01	Configuração inicial do GitHub Projects v2, definição das colunas Backlog → To Do → Doing → In review → Done, definição do limite de WIP igual a 3 e implementação da consulta GraphQL inicial para 100 repositórios.	Trio	#1 (RQ01/RQ02, João Gabriel), #2 (RQ03/RQ04, Lucas Ferreira), #3 (RQ05/RQ06, Vítor Veiga), #13 (RQ07/Bônus, Lucas Ferreira)
Lab01S02	Implementação da paginação por cursor para a coleta dos 1.000 repositórios, exportação do dataset em formato .csv, verificação dos dados e elaboração das hipóteses informais. Também foi realizado o primeiro snapshot do quadro.	Trio	#4 (Paginação RQ01/RQ02, João Gabriel), #5 (Paginação RQ03/RQ04, Vítor Veiga), #6 (Paginação RQ05/RQ06, Lucas Ferreira)
Lab01S03	Limpeza dos dados, processamento estatístico e criação das visualizações para as RQs e para a RQ08.	Trio	#7 (Visualização RQ01/RQ02, João Gabriel), #8 (Visualização RQ03/RQ04, Lucas Ferreira), #9 (Visualização RQ05/RQ06, Vítor Veiga)
Relatório Final	Organização e redação do relatório, comparação entre as hipóteses e os resultados encontrados, documentação da RQ08 e consolidação das informações do processo.	Trio	#10

O fluxo de trabalho foi acompanhado pelo GitHub Projects v2, que ficou vinculado ao repositório do grupo. Cada atividade foi registrada como uma Issue e atribuída a um dos integrantes. A utilização das Issues também permitiu acompanhar a execução das tarefas e relacioná-las aos commits realizados durante o desenvolvimento.

Ao final do laboratório, o quadro Kanban apresentava o fluxo de atividades entre as colunas Backlog, To Do, Doing, In review e Done, respeitando o limite de três tarefas simultâneas na coluna Doing.

Laboratório de Experimentação de Software

Add status update

Insights

Workflows 7

...

Backlog

Priority board

Team items

Roadmap

My items

New view

Filter by keyword or by field

Backlog 3/9 Estimate: 0

This item hasn't been started

Laborat-rio-de-Experimenta-o-de-Software #7 Lab01503 - Analise e visualização (RQ01 e RQ02)

Laborat-rio-de-Experimenta-o-de-Software #8 Lab01503 - Analise e visualização (RQ03 e RQ04)

Laborat-rio-de-Experimenta-o-de-Software #9 Lab01503 - Analise e visualização (RQ05, RQ06)

+ Add item

To Do 3 Estimate: 0

This is ready to be picked up

Laborat-rio-de-Experimenta-o-de-Software #5 Lab01502 - Paginação e validação das RQ03 e RQ04

Laborat-rio-de-Experimenta-o-de-Software #6 Lab01502 - Paginação e validação das RQ05, RQ06

Laborat-rio-de-Experimenta-o-de-Software #4 Lab01502 - Paginação e validação das RQ01 e RQ02

Doing 0/3 Estimate: 0

This is actively being worked on

In review 0/5 Estimate: 0

This item is in review

Done 4 Estimate: 0

This has been completed

Laborat-rio-de-Experimenta-o-de-Software #1 Lab01501 - Implementar consulta GraphQL (RQ01 e RQ02)

Laborat-rio-de-Experimenta-o-de-Software #2 Lab01501 - Implementar consulta GraphQL (RQ03 e RQ04)

Laborat-rio-de-Experimenta-o-de-Software #10

Laborat-rio-de-Experimenta-o-de-Software #11 Lab01501 - Implementar consulta GraphQL (RQ05, RQ06)

Laborat-rio-de-Experimenta-o-de-Software #13 Lab01501 - Implementar consulta GraphQL (RQ07 bonus)

Laboratório de Experimentação de Software

Add status update

Insights

Workflows 7

...

Backlog

Priority board

Team items

Roadmap

My items

New view

Filter by keyword or by field

Backlog 0/9 Estimate: 0

This item hasn't been started

To Do 3 Estimate: 0

This is ready to be picked up

Laborat-rio-de-Experimenta-o-de-Software #7 Lab01503 - Analise e visualização (RQ01 e RQ02)

Laborat-rio-de-Experimenta-o-de-Software #8 Lab01503 - Analise e visualização (RQ03 e RQ04)

Laborat-rio-de-Experimenta-o-de-Software #9 Lab01503 - Analise e visualização (RQ05, RQ06)

Doing 0/3 Estimate: 0

This is actively being worked on

In review 0/5 Estimate: 0

This item is in review

Done 7 Estimate: 0

This has been completed

Laborat-rio-de-Experimenta-o-de-Software #5 Lab01502 - Paginação e validação das RQ03 e RQ04

Laborat-rio-de-Experimenta-o-de-Software #6 Lab01502 - Paginação e validação das RQ05, RQ06

Laborat-rio-de-Experimenta-o-de-Software #4 Lab01502 - Paginação e validação das RQ01 e RQ02

Laborat-rio-de-Experimenta-o-de-Software #1 Lab01501 - Implementar consulta GraphQL (RQ01 e RQ02)

Laborat-rio-de-Experimenta-o-de-Software #2 Lab01501 - Implementar consulta GraphQL (RQ03 e RQ04)

Laborat-rio-de-Experimenta-o-de-Software #3 Lab01501 - Implementar consulta GraphQL (RQ05, RQ06)

Laborat-rio-de-Experimenta-o-de-Software #11

Laborat-rio-de-Experimenta-o-de-Software #10

JoaoAlmeida02 / Projects / Laboratório de Experimentação de Software

Type to search

Insights

...

Backlog

Priority board

Team items

Roadmap

My items

Filter by keyword or by field

Backlog 0/3 Estimate: 0

This item hasn't been started

To Do 0 Estimate: 0

This is ready to be picked up

Doing 0/3 Estimate: 0

This is actively being worked on

In review 0/5 Estimate: 0

This item is in review

Done 15 Estimate: 0

This has been completed

Laborat-rio-de-Experimenta-o-de-Software #21 Lab01503 - Relatório final

Laborat-rio-de-Experimenta-o-de-Software #9 Lab01503 - Analise e visualização (RQ05, RQ06)

Laborat-rio-de-Experimenta-o-de-Software #7 Lab01503 - Analise e visualização (RQ01 e RQ02)

Laborat-rio-de-Experimenta-o-de-Software #8 Lab01503 - Analise e visualização (RQ03 e RQ04)

Laborat-rio-de-Experimenta-o-de-Software #6 Lab01502 - Paginação e validação das RQ03 e RQ04

Laborat-rio-de-Experimenta-o-de-Software #5 Lab01502 - Paginação e validação das RQ05, RQ06

Laborat-rio-de-Experimenta-o-de-Software #4

3.4 Ferramentas

Para realizar a coleta e o processamento dos dados, foram utilizadas as seguintes ferramentas:

- GitHub GraphQL API (v4): utilizada para realizar a coleta dos dados dos 1.000 repositórios mais estrelados, utilizando a busca stars:>1 sort:stars-desc.
- Python (3.10+): utilizada no desenvolvimento do script responsável pela coleta, paginação e processamento dos dados.
- Módulos nativos do Python (csv, os, sys, time, statistics, collections e datetime): utilizados para tarefas como manipulação dos arquivos, controle do tempo entre requisições, tratamento das datas e cálculo das métricas estatísticas.
- Requests e Python-dotenv: o módulo Requests foi utilizado para realizar as requisições HTTP à API GraphQL. O Python-dotenv foi utilizado para carregar o Personal Access Token por meio de uma variável de ambiente no arquivo .env.
- GitHub Projects (v2): utilizado para organizar as tarefas do grupo, acompanhar o andamento das atividades e controlar o limite de WIP.

3.5 Tabela de Métricas

RQ	Métrica	Definição Operacional	Unidade	Ferramenta / Fonte
RQ01	Idade do repositório	Diferença entre a data atual e a data de criação do repositório (createdAt).	Dias e anos	Script GraphQL (createdAt)
RQ02	Contribuição externa	Quantidade total de pull requests que foram mescladas.	Contagem inteira	Script GraphQL (pullRequests(states: MERGED))
RQ03	Frequência de releases	Quantidade total de releases registradas no repositório.	Contagem inteira	Script GraphQL (releases.totalCount)
RQ04	Tempo até a última atualização	Diferença entre a data atual e a data da última atualização (pushedAt).	Dias	Script GraphQL (pushedAt)
RQ05	Linguagem primária	Linguagem principal identificada pelo GitHub para cada repositório.	Categoria nominal	Script GraphQL (primaryLanguage.name)
RQ06	Razão de issues fechadas	Relação entre o número de issues fechadas e o total de issues abertas e fechadas.	Razão de 0 a 1	Script GraphQL (issuesOpen e issuesClosed)
RQ07	Métricas por linguagem	Comparação das métricas das RQs 02, 03 e 04 de acordo com a linguagem primária do repositório.	Variada	Script Python / GitHub Octoverse 2025

RQ08	Razão Estrelas/Forks	Relação entre o número de estrelas e o número de forks do repositório.	Razão numérica	Script GraphQL (stargazerCount e forkCount)
------	----------------------	--	----------------	---

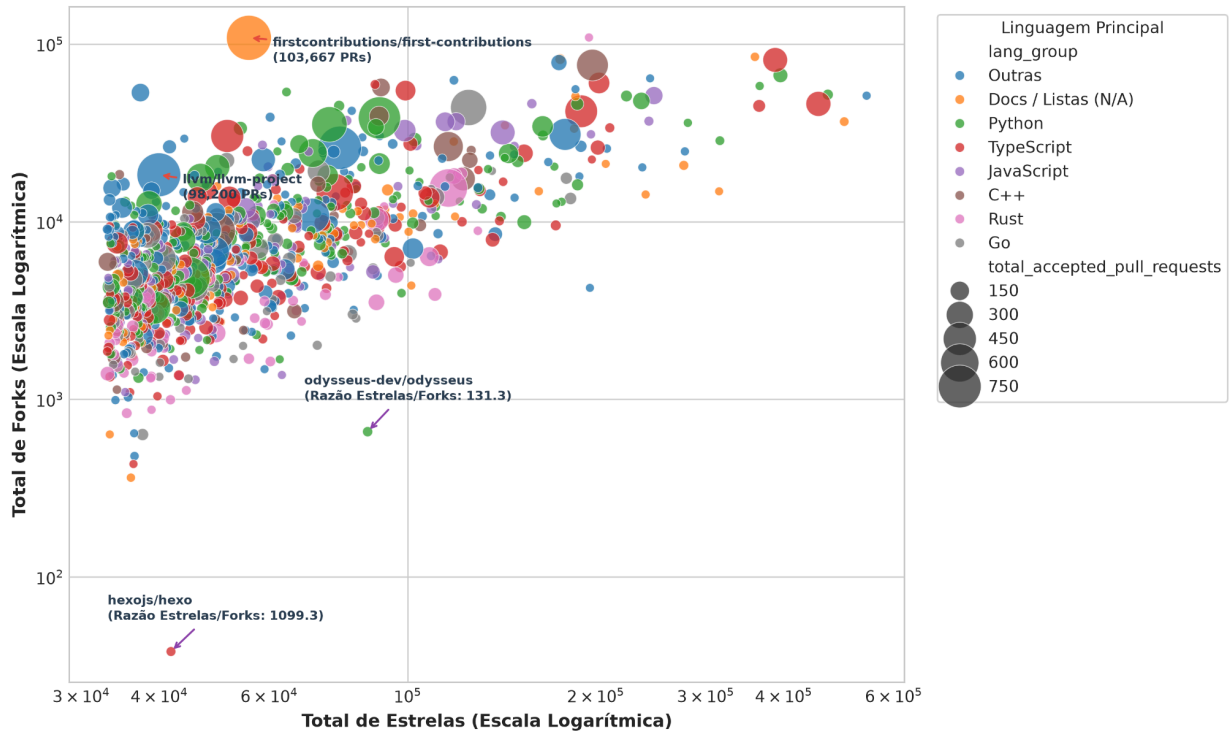
3.6 Inovações Propostas pelo Grupo

Como parte da proposta de inovação do laboratório, o grupo adicionou uma questão de pesquisa própria, além de algumas melhorias no processo de coleta dos dados.

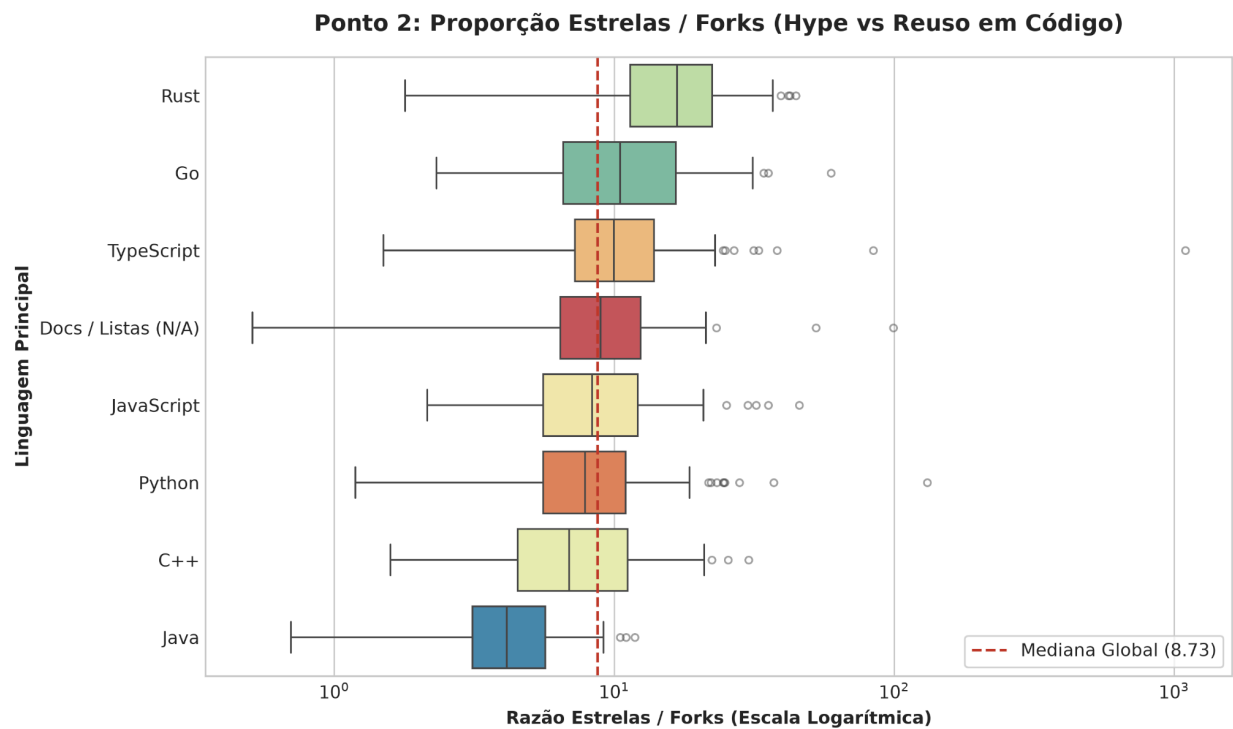
A primeira inovação foi a criação da RQ08, que busca analisar se o número de estrelas realmente representa o nível de engajamento dos projetos ou se está mais relacionado à popularidade. A ideia é comparar o número de estrelas com o número de forks. As estrelas podem representar apenas o interesse de um usuário pelo projeto, enquanto o fork indica uma ação mais direta de utilização ou desenvolvimento a partir daquele código. Dessa forma, a razão entre estrelas e forks pode ajudar a identificar diferenças entre projetos que possuem grande popularidade e aqueles que apresentam maior nível de reuso.

Para analisar esse comportamento visualmente, o grupo gerou dois gráficos específicos. O primeiro deles (Ponto 1: Hype vs. Reuso Real — Estrelas × Forks × PRs Aceitos) apresenta uma dispersão em escala logarítmica que cruza estrelas e forks, utilizando o tamanho dos pontos para indicar o total de PRs aceitos e as cores para identificar a linguagem principal. Nessa visualização, projetos consolidados e com alto volume de contribuições, como `firstcontributions/first-contributions` (103.667 PRs) e `llvm/llvm-project` (98.200 PRs), aparecem com grande volume de reuso e colaboração ativa. Em contrapartida, repositórios como `hexojs/hexo` e `odysseus-dev/odysseus` destacam-se com uma razão de estrelas por fork bastante elevada (1099,3 e 131,3, respectivamente), indicando forte acúmulo de estrelas passivas em relação ao número de forks criados.

Ponto 1: Hype vs. Reuso Real — Estrelas x Forks x PRs Aceitos



O segundo gráfico (Ponto 2: Proporção Estrelas / Forks por Linguagem) traz um boxplot que compara a distribuição dessa razão entre as principais linguagens e materiais de documentação, tendo como referência a linha da mediana global de 8,73. Nele, observa-se que linguagens tradicionais de sistemas corporativos, como Java e C++, apresentam medianas menores de estrelas por fork, refletindo um reuso proporcionalmente mais próximo ao volume de estrelas. Já ecossistemas como Rust, Go e TypeScript exibem medianas mais altas, apontando maior tração de visibilidade e estrelas na plataforma quando comparados à taxa de bifurcação do código.



Além da RQ08 e de suas análises gráficas, o grupo também trabalhou na melhoria do processo de coleta utilizando a API GraphQL. A consulta foi estruturada para obter, em uma mesma requisição, diferentes informações necessárias para as análises, como pull requests mescladas, releases, issues abertas e fechadas, além do número de forks.

O script também utiliza paginação por cursor para percorrer os resultados e possui mecanismos de tentativa novamente em caso de falhas temporárias. Foi utilizado um sistema de backoff progressivo para lidar com erros de rede, respostas do servidor e situações relacionadas ao limite de requisições da API. Essas medidas ajudaram a tornar a coleta mais estável e reduzir a possibilidade de perda de dados durante a análise dos 1.000 repositórios.

4. Resultados

4.1 Coleta de Dados

A coleta realizada por meio da API GraphQL do GitHub reuniu os dados dos 1.000 repositórios com maior número de estrelas, utilizando a consulta `stars:>1 sort:stars-desc`.

Dos 1.000 repositórios analisados, 86 (8,6%) não possuíam uma linguagem primária identificada (`primaryLanguage == null`). Esses projetos eram, em sua maioria, voltados para documentação, livros, tutoriais e listas de recursos, como as chamadas awesome-lists.

Também foram encontrados 44 repositórios (4,4%) sem nenhuma issue aberta ou fechada. Nesses casos, não foi possível calcular a razão de issues fechadas, e o resultado foi registrado como N/A.

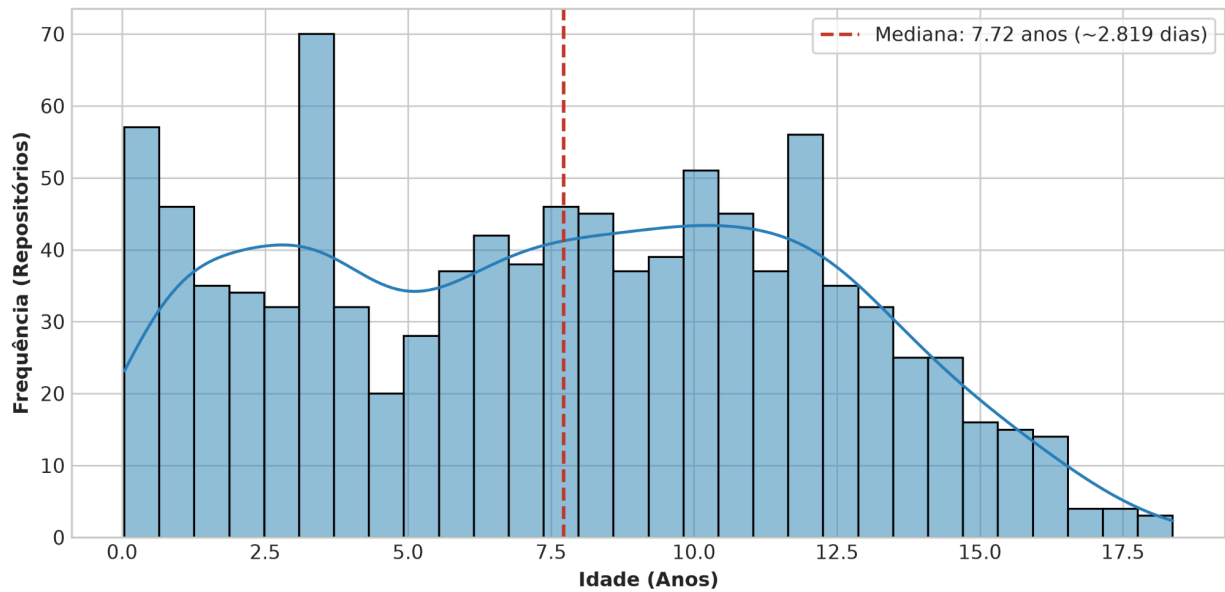
Em relação aos forks, todos os 1.000 repositórios apresentaram valores válidos. A quantidade variou de 38 a 108.909 forks. Dessa forma, foi possível calcular a razão entre estrelas e forks para toda a amostra.

4.2 Visualização Gráfica

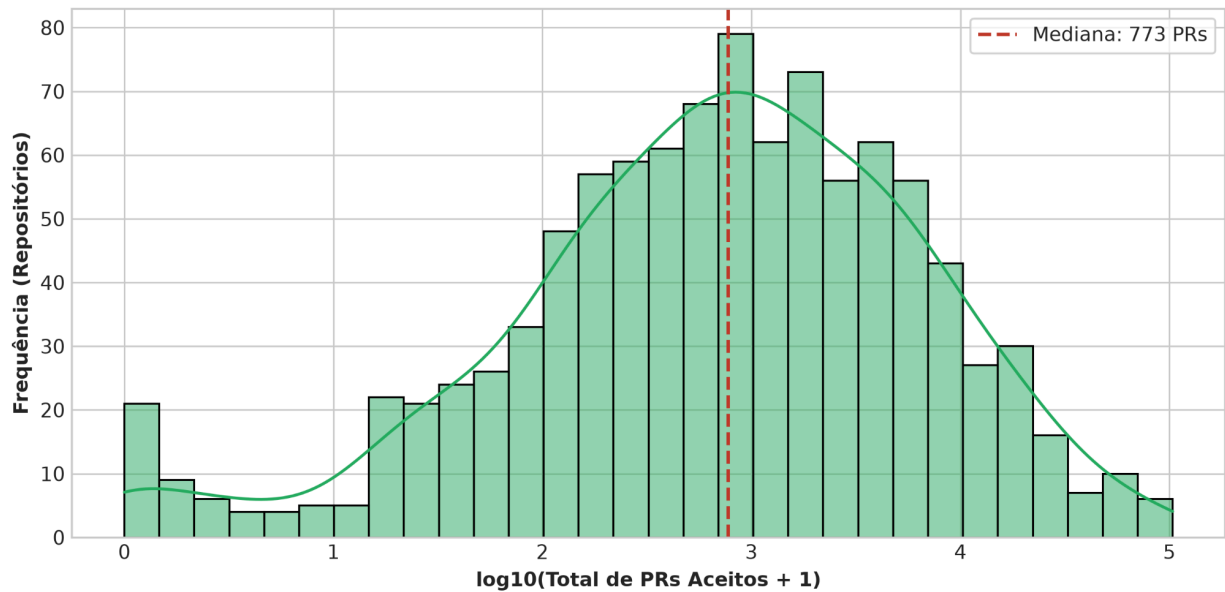
Questão de Pesquisa	Métrica Principal	Mediana	Média	Mínimo	Máximo
RQ01	Idade do repositório (dias / anos)	2.819 dias (~7,72 anos)	2.788,49 dias	13 dias	6.711 dias
RQ02	Pull Requests aceitos (MERGED)	773 PRs	4.264,63 PRs	0	103.667
RQ03	Total de releases catalogadas	41 releases	128,80 releases	0	1.000
RQ04	Dias até a última atualização	1 dia	111,57 dias	0 dias	2.458 dias
RQ05	Linguagem primária predominante	Python (22,8%)	-	-	-
RQ06	Razão de issues fechadas / total	0,88 (88,0%)	0,80 (80,0%)	0,075	1,000
RQ08 (Bônus)	Razão Estrelas / Forks	8,73	11,87	0,51	1.099,29

Imagens dos gráficos

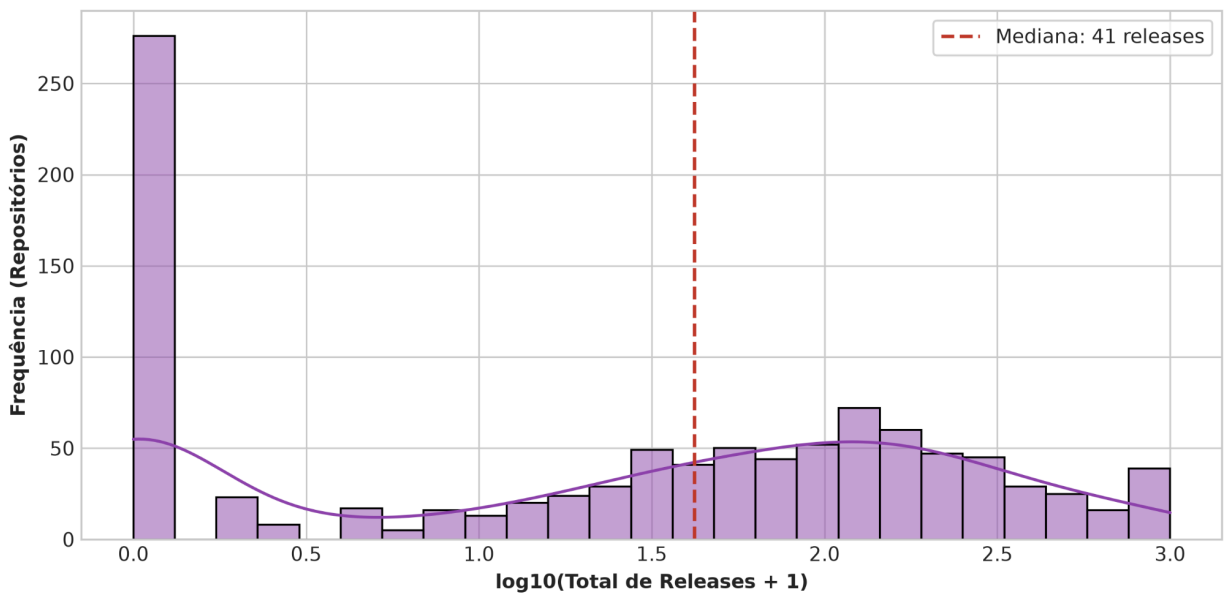
RQ 01. Sistemas populares são maduros/antigos?
Distribuição da Idade dos Repositórios



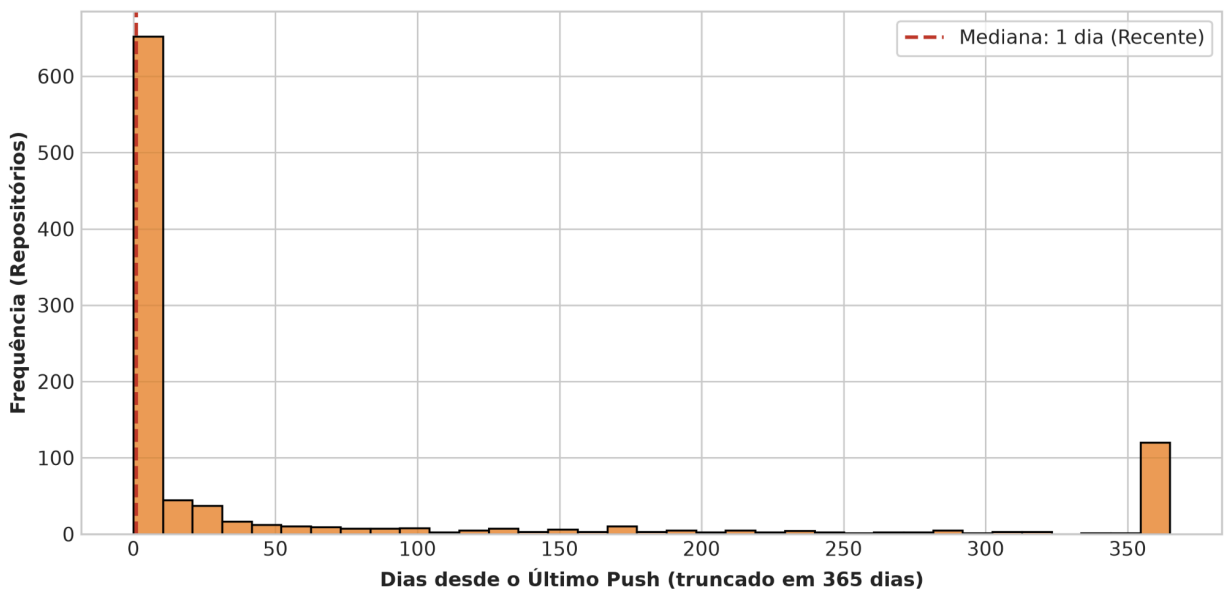
RQ 02. Sistemas populares recebem muita contribuição externa?
Distribuição de PRs Aceitos (Escala Log10)



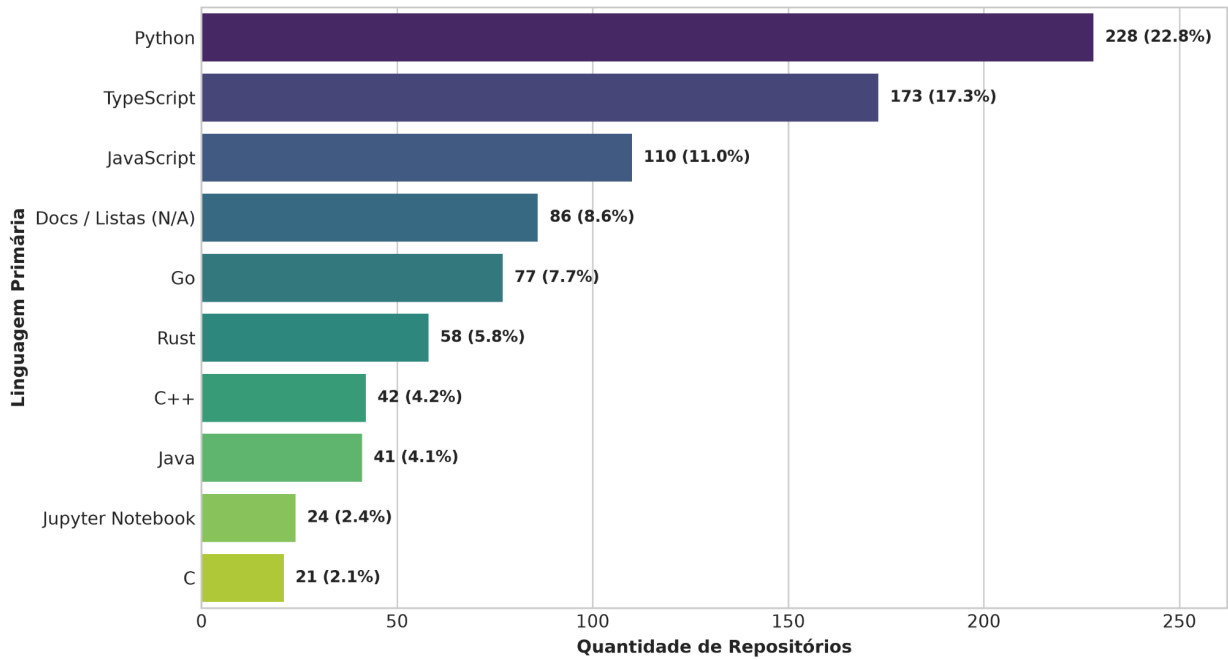
RQ 03. Sistemas populares lançam releases com frequência?
Distribuição de Total de Releases (Escala Log10)



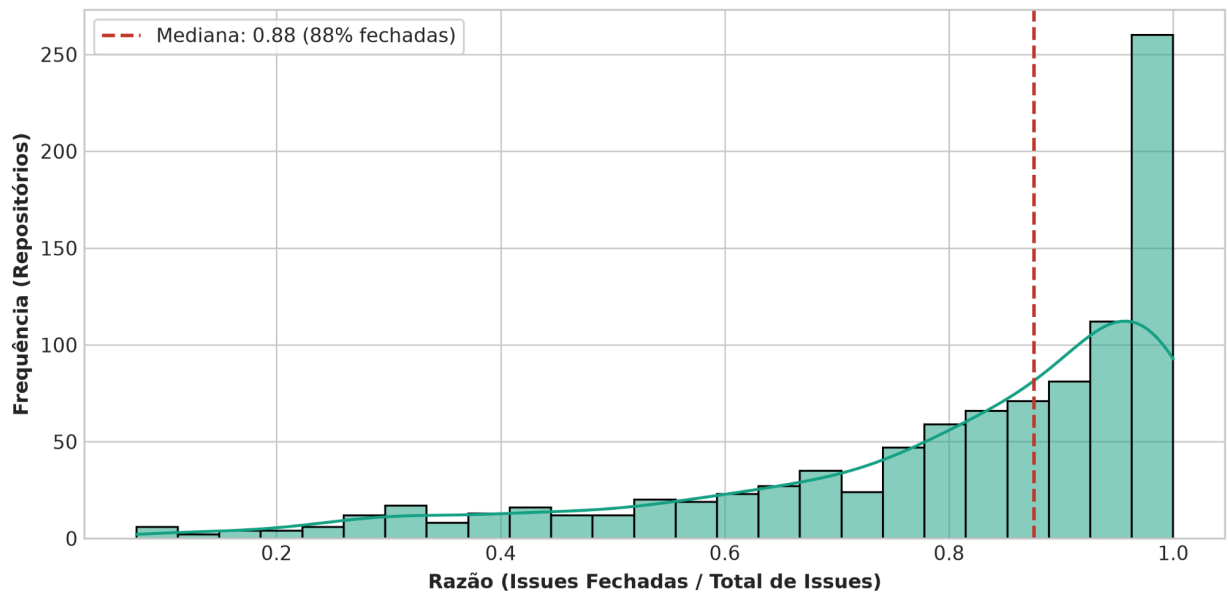
RQ 04. Sistemas populares são atualizados com frequência?
Distribuição de Dias desde a Última Atualização



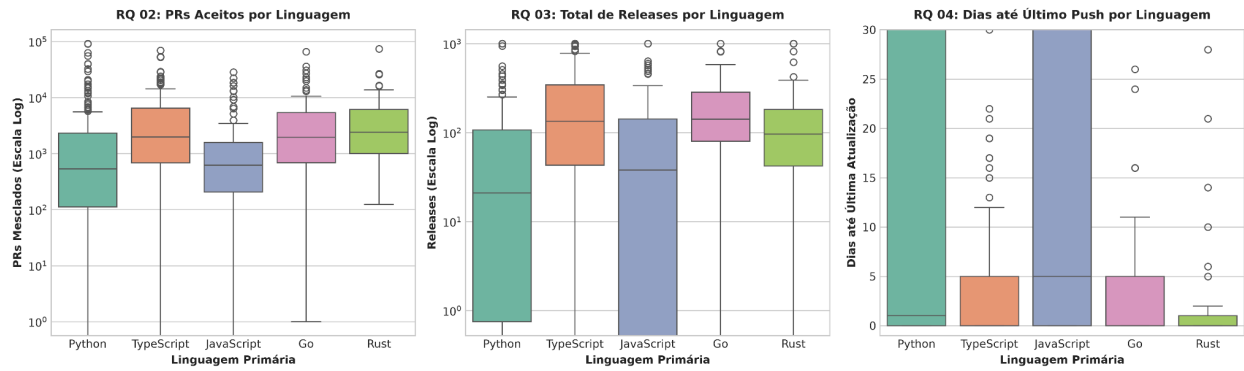
RQ 05. Sistemas populares são escritos nas linguagens mais populares?
Top 10 Linguagens Primárias (com base no GitHub Octoverse 2025)



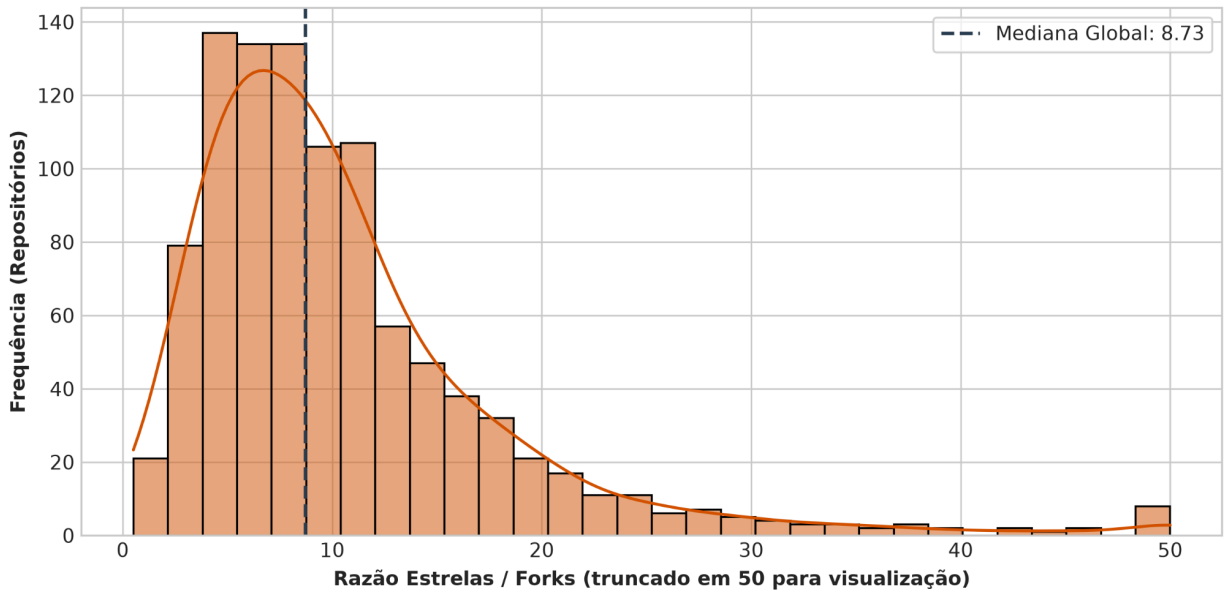
RQ 06. Sistemas populares possuem um alto percentual de issues fechadas?
Distribuição da Taxa de Resolução de Issues



RQ 07. Sistemas escritos em linguagens mais populares recebem mais contribuição externa, lançam mais releases e são atualizados com mais frequência?



RQ 08 (Bônus). Estrelas correlacionam com engajamento real ou só fama?
Distribuição da Razão Estrelas/Forks (Endosso Passivo vs Reuso)



4.3 Discussão

1. RQ01. Sistemas populares são maduros/antigos?

- **Confronto com a Hipótese H1:** Confirmada. A mediana de 2.819 dias, aproximadamente 7,72 anos, indica que a maioria dos repositórios mais populares possui uma trajetória relativamente longa. Esse resultado apoia a ideia de que projetos de grande destaque no GitHub geralmente precisam de tempo para construir uma comunidade e alcançar maior visibilidade. Apesar disso, também foram encontrados projetos recentes, com o menor valor sendo de 13 dias, mostrando que alguns projetos conseguem crescer rapidamente.

2. RQ02. Sistemas populares recebem muita contribuição externa?

- **Confronto com a Hipótese H2:** Confirmada. A mediana de 773 PRs mescladas mostra que os repositórios mais populares recebem uma quantidade significativa de contribuições externas. A média foi de 4.264,63 PRs, enquanto o maior valor chegou a 103.667. A diferença entre esses valores mostra que os dados possuem uma grande variação e que alguns projetos concentram uma quantidade muito maior de contribuições.

3. RQ03. Sistemas populares lançam releases com frequência?

- **Confronto com a Hipótese H3:** Parcialmente confirmada. A mediana de 41 releases indica que muitos dos projetos analisados possuem uma frequência considerável de lançamentos. Porém, alguns repositórios não possuem nenhuma release formal. Isso pode acontecer principalmente em projetos educacionais, listas de recursos ou projetos que utilizam outras formas de disponibilizar atualizações.

4. RQ04. Sistemas populares são atualizados com frequência?

- **Confronto com a Hipótese H4:** Confirmada. A mediana de apenas 1 dia desde a última atualização (pushedAt) mostra que grande parte dos repositórios continua recebendo alterações com frequência. A média de 111,57 dias é bem maior devido à presença de alguns projetos que estão há bastante tempo sem atualizações, mas que ainda possuem muitas estrelas acumuladas.

5. RQ05. Sistemas populares são escritos nas linguagens mais populares?

- **Confronto com a Hipótese H5:** Confirmada. As linguagens mais presentes na amostra foram Python, com 22,8%, TypeScript, com 17,3%, e JavaScript, com 11,0%. Juntas, elas representam 51,1% dos repositórios analisados. Esse resultado está de acordo com o destaque dessas linguagens no GitHub Octoverse 2025 e mostra a presença significativa de projetos relacionados à Inteligência Artificial, desenvolvimento web e aplicações full-stack.

6. RQ06. Sistemas populares possuem um alto percentual de issues fechadas?

- **Confronto com a Hipótese H6:** Confirmada. A mediana da razão de issues fechadas foi de 0,88, ou seja, 88%. Esse valor ficou acima do limite esperado na hipótese, que era de pelo menos 70%. O resultado indica que, na maior parte dos projetos analisados, existe uma proporção elevada de issues encerradas em relação ao total de issues.

7. RQ07. Sistemas escritos em linguagens mais populares recebem mais contribuição externa, lançam mais releases e são atualizados com mais frequência?

- **Confronto com a Hipótese H7:** Confirmada. Os projetos desenvolvidos em Python, TypeScript e JavaScript apresentaram, de forma geral, maior volume de PRs mescladas e maior frequência de atualizações quando comparados a projetos desenvolvidos em linguagens menos presentes na amostra. Esse resultado sugere uma relação entre a popularidade das linguagens e o nível de atividade dos repositórios.

8. RQ08. Estrelas correlacionam com engajamento real, ou representam principalmente popularidade?

- **Confronto com a Hipótese H8:** Confirmada. A mediana da razão entre estrelas e forks foi de 8,73, enquanto o maior valor chegou a 1.099,29. Isso mostra que, em muitos dos repositórios analisados, o número de estrelas é muito maior que o número de forks. O resultado sugere que as estrelas podem representar principalmente interesse ou popularidade, enquanto o fork está mais relacionado ao uso e ao reaproveitamento efetivo do código.

5. Conclusão

A análise dos 1.000 repositórios mais populares do GitHub mostrou que a popularidade está, em grande parte, relacionada a projetos com alguns anos de existência e que continuam recebendo atualizações. A mediana de idade foi de aproximadamente 7,7 anos, enquanto o tempo desde o último push teve mediana de apenas 1 dia. Também foi observada uma taxa elevada de issues fechadas, com mediana de 88%, indicando que grande parte dos projetos mantém um fluxo ativo de resolução de problemas.

As linguagens mais presentes na amostra foram Python, TypeScript e JavaScript, que juntas representam uma parcela significativa dos repositórios analisados. Esse resultado acompanha a importância dessas linguagens no cenário atual do desenvolvimento de software.

A RQ08 também trouxe um resultado importante para a análise. A comparação entre estrelas e forks mostrou que o número de estrelas nem sempre representa um alto nível de participação técnica. Em muitos casos, os repositórios possuem muitas estrelas, mas uma quantidade bem menor de forks. Isso indica que as estrelas podem estar mais relacionadas à popularidade e ao interesse pelo projeto do que ao uso ou à contribuição direta no código.

O estudo possui algumas limitações. A coleta foi realizada em um único momento, portanto os resultados representam apenas uma fotografia do estado dos repositórios. Além disso, a amostra inclui projetos que não são necessariamente softwares executáveis, como listas de recursos, tutoriais e materiais educacionais. Isso pode influenciar algumas métricas, principalmente as relacionadas a linguagens e releases.

Como possibilidade para trabalhos futuros, seria interessante separar os repositórios de acordo com seu tipo, diferenciando projetos de software executável de materiais informativos. Outra possibilidade seria realizar novas coletas ao longo do tempo para acompanhar como as métricas de popularidade, contribuição e manutenção evoluem.

5. Referências

- ZUSE, Horst. A framework of software measurement. Walter de Gruyter, 2013.
- GITHUB. **Octoverse: A new developer joins GitHub every second as AI leads TypeScript to #1.** GitHub Blog, 28 out. 2025. Disponível em: <https://github.blog/news-insights/octoverse/octoverse-a-new-developer-joins-github-every-second-as-ai-leads-typescript-to-1/>. Acesso em: 2026.